

Universidade Federal de Alfenas – UNIFAL-MG

Instituto de Ciências Exatas – ICEx

Bacharelado em Ciência da Computação

Relatório Técnico – Trabalho Final

Implementação do Protocolo Go-Back-N em Java via UDP

Disciplina: Redes de Computadores

Aluno: Gean Marques

Professor: Flavio Barbieri Gonzaga

Alfenas, julho de 2026

1. Introdução

Este relatório documenta as decisões de projeto, as dificuldades encontradas, os testes realizados e a análise dos resultados obtidos na implementação do protocolo de transferência confiável **Go-Back-N (GBN)** sobre o protocolo UDP em Java. O trabalho foi realizado como atividade final da disciplina de Redes de Computadores da UNIFAL-MG, com base no referencial teórico do Capítulo 3 do livro *Redes de Computadores: Uma Abordagem Top-Down* (Kurose & Ross, 8ª edição).

O objetivo central foi implementar dois módulos Java independentes — **Emissor** e **Receptor** — que se comunicam exclusivamente por meio de sockets UDP (`DatagramSocket` / `DatagramPacket`), sem qualquer framework externo além do JDK padrão. Como o UDP não oferece garantias de entrega, ordenação ou controle de fluxo, toda a lógica de confiabilidade precisou ser construída na camada de aplicação, replicando fielmente as máquinas de estados finitas (FSMs) descritas nas Figuras 3.20 e 3.21 do livro-texto.

2. Decisões de Projeto

2.1 Formato do Datagrama

O cabeçalho de cada datagrama UDP foi projetado conforme a especificação do enunciado (Seção 3.4) e serializado com `ByteBuffer` para garantir portabilidade entre plataformas. Os campos fixos totalizam **11 bytes**:

Campo	Tamanho	Descrição
tipo	1 byte	0=DATA, 1=ACK, 2=HANDSHAKE, 3=FIN
num_seq	4 bytes (int)	Número de sequência do pacote
num_ack	4 bytes (int)	Confirmação cumulativa (em ACKs)
tamanho_dados	2 bytes (short)	Bytes válidos no payload
dados	até 1024 bytes	Payload do arquivo

Pacotes de controle HANDSHAKE reaproveitam o campo `dados` para transportar uma string delimitada por '|' com os parâmetros da sessão: caminho de destino, tamanho do arquivo, probabilidade de perda e hash MD5 do arquivo original. ACKs de controle usam sentinelas negativas no campo `num_ack`: -1 para confirmação do handshake e -2 para confirmação do FIN.

2.2 Modelo de Threads e Sincronização

O Emissor utiliza **duas threads concorrentes**: a thread principal envia segmentos respeitando a janela deslizante, bloqueando com `wait()/notifyAll()` quando a janela está cheia; uma thread dedicada (`ACK-Listener`) escuta ACKs e avança a variável `base`. O acesso às variáveis compartilhadas é protegido por um único monitor (`lock`), evitando condições de corrida sem overhead desnecessário.

2.3 Temporizador Único

Conforme a FSM do Kurose & Ross (Fig. 3.20), implementou-se **um único** `java.util.Timer` associado ao pacote mais antigo não confirmado (`base`). O timer é cancelado quando `base ==`

`nextSeqNum` é reiniciado a cada ACK que avança a janela ou a cada timeout, que retransmite **todos** os pacotes do intervalo $[base, nextSeqNum - 1]$ — comportamento central do GBN.

2.4 Buffer Circular

Os pacotes em trânsito são armazenados em um array de tamanho N indexado por $seq \% N$ (buffer circular), eliminando a necessidade de reler o arquivo do disco a cada retransmissão. Isso reduz latência de I/O e simplifica a implementação da janela deslizando.

2.5 Handshake e FIN Confiáveis

Por serem pacotes únicos de controle, o HANDSHAKE e o FIN possuem lógica própria de confirmação com retentativas (até 5 tentativas e timeouts de 2 s e 1,5 s, respectivamente). Eles não são submetidos à perda simulada — que incide apenas sobre pacotes DATA recebidos em ordem, conforme a Seção 4 do enunciado.

2.6 Verificação de Integridade (MD5)

Ao final de cada sessão o Receptor recalcula o hash MD5 do arquivo salvo e compara com o hash enviado pelo Emissor no handshake, usando `MessageDigest` da API Java padrão. Em todos os 24 testes realizados, a verificação retornou **"OK (hashes idênticos)"**, confirmando a transferência bit a bit correta.

2.7 Timeout de Inatividade no Receptor

Uma dificuldade prática encontrada durante os testes foi o Receptor ficar bloqueado indefinidamente caso o Emissor fosse encerrado abruptamente no meio de uma transferência. Para contornar isso, adicionou-se um timeout de inatividade de 30 s: se nenhum pacote chegar nesse intervalo, a sessão corrente é abandonada e o Receptor volta a aguardar um novo handshake — recurso importante para robustez durante a apresentação ao vivo.

3. Dificuldades Encontradas

Condições de corrida: A interação entre a thread que envia pacotes e a thread ACK-Listener exigiu cuidado especial. Em versões iniciais, a variável `base` era lida sem sincronização adequada, o que causava envios além da janela ou timeouts espúrios. A solução foi centralizar todo o estado mutável em um único monitor e usar `wait()/notifyAll()` de forma consistente.

Disparo prematuro do timer: Quando um ACK chegava ao mesmo tempo em que o timer expirava, era possível que o timeout tratasse retransmissões já desnecessárias. Isso foi resolvido verificando, dentro de `onTimeout()`, se `base >= nextSeqNum` antes de qualquer retransmissão.

Encoding do console: O ambiente Windows utilizava locale ANSI (CP1252), enquanto o código gerava saídas em UTF-8. Acentuação era corrompida nos logs. Resolvido configurando `System.setOut(new PrintStream(System.out, true, StandardCharsets.UTF_8))` no início de cada `main()`.

Contagem de pacotes fora de ordem: Em janelas grandes com perda, um único pacote perdido faz o Emissor retransmitir até N pacotes; todos os retransmitidos fora do `expectedSeqNum` corrente chegam ao Receptor como 'fora de ordem'. Compreender essa relação foi fundamental para interpretar corretamente as estatísticas do Receptor.

4. Testes Realizados

Os testes foram conduzidos em modo **localhost** (Emissor e Receptor na mesma máquina Windows 11, via Git Bash), com o arquivo de teste de **1 MB** (1024 segmentos de 1024 bytes) e perda simulada exclusivamente pelo Receptor, conforme a Seção 4 do enunciado. O script `testes/executar_testes.sh` automatizou 24 combinações (6 valores de $N \times 4$ probabilidades de perda), com resultados exportados para CSV e confirmados individualmente nos logs de cada execução.

Valores de N testados: **1, 2, 4, 8, 16, 32**. Probabilidades de perda: **0%, 5%, 10%, 20%**. Além da bateria automatizada, realizou-se manualmente um teste de **arquivo menor que um segmento** (48 bytes) e testes de validação de argumentos inválidos no Emissor.

Em **todos os 24 cenários**, a verificação de integridade MD5 confirmou que o arquivo recebido era idêntico ao original, demonstrando corretude do protocolo implementado.

5. Análise dos Resultados

5.1 Tabela de Resultados Completos

A tabela a seguir consolida todas as métricas coletadas pelo Emissor e pelo Receptor para cada combinação testada. Os valores de throughput foram extraídos diretamente dos logs individuais de cada execução.

N	p (%)	Tempo (s)	Enviados	Retransmissões	ACKs recv.	Throughput (KB/s)	Perda efetiva (%)
1	0%	0.202	1024	0	1024	5.069,31	0.00%
	5%	62.249	1086	62	1024	16.45	5.71%
	10%	113.272	1137	113	1024	9.04	9.94%
	20%	287.542	1311	287	1024	3.56	21.89%
2	0%	0.199	1024	0	1024	5.145,73	0.00%
	5%	52.225	1128	104	1076	19.61	4.83%
	10%	99.244	1222	198	1123	10.32	8.82%
	20%	298.356	1620	596	1322	3.43	22.54%
4	0%	0.264	1024	0	1024	3.878,79	0.00%
	5%	49.210	1220	196	1171	20.81	4.57%
	10%	122.249	1512	488	1390	8.38	10.65%
	20%	239.340	1977	953	1738	4.28	18.92%
8	0%	0.167	1024	0	1024	6.131,74	0.00%
	5%	56.393	1472	448	1416	18.16	5.19%
	10%	119.029	1968	944	1850	8.60	10.33%
	20%	297.529	3368	2344	3066	3.44	22.37%
16	0%	0.175	1024	0	1024	5.851,43	0.00%

N	p (%)	Tempo (s)	Enviados	Retransmissões	ACKs recv.	Throughput (KB/s)	Perda efetiva (%)
	5%	51.641	1840	816	1789	19.83	4.74%
	10%	131.292	3102	2078	2972	7.80	11.27%
	20%	258.180	5093	4069	4837	3.97	20.00%
32	0%	0.311	1024	0	1024	3.292,60	0.00%
	5%	58.665	2856	1832	2798	17.46	5.36%
	10%	110.289	4477	3453	4367	9.28	9.70%
	20%	251.402	8954	7930	8703	4.07	19.69%

5.2 Transferência sem Perda (p = 0%)

Com probabilidade de perda nula, o protocolo funciona como pipeline simples. Os tempos foram todos inferiores a 350 ms para 1 MB, sem nenhuma retransmissão.

Janela (N)	Tempo (ms)	Throughput (KB/s)
1	202	5.069,31
2	199	5.145,73
4	264	3.878,79
8	167	6.131,74
16	175	5.851,43
32	311	3.292,60

Os tempos com $p=0$ são notavelmente similares entre todos os valores de N (167 ms a 311 ms). Isso é esperado: em **rede local (localhost)**, a latência de propagação é virtualmente nula e a capacidade do enlace não é o gargalo. O protocolo flui livremente, e o tempo total é dominado pelo overhead de processamento e de I/O local. O throughput aparente varia de 3.292,60 a 6.131,74 KB/s — variação atribuída a flutuações normais do escalonador do SO e não a diferenças fundamentais entre os valores de N.

Um resultado contraintuitivo é que $N=32$ apresenta *menor* throughput (3.292,60 KB/s) que $N=8$ (6.131,74 KB/s) mesmo sem perda. Isso é causado pelo overhead de gerenciamento do timer e de notificações entre threads: com janela maior, o emissor envia rafagas maiores que ocasionalmente disputam recursos com a thread ACK-Listener, introduzindo brevíssimas esperas que acumulam no tempo total.

5.3 Impacto da Probabilidade de Perda

A degradação de desempenho com o aumento da taxa de perda é drástica e diretamente ligada ao mecanismo do GBN. A tabela abaixo mostra o número de retransmissões por combinação (N, p):

$p \rightarrow N \downarrow$	$N=1$	$N=2$	$N=4$	$N=8$	$N=16$	$N=32$
$p=5\%$	62	104	196	448	816	1832
$p=10\%$	113	198	488	944	2078	3453
$p=20\%$	287	596	953	2344	4069	7930

Observa-se que as retransmissões crescem **aproximadamente na proporção $N \times \text{número_de_perdas}$** : com $N=32$ e $p=20\%$, foram 7.930 retransmissões para ~251 perdas primárias, uma razão de ~31,6 — próxima de N . Isso reflete o princípio do GBN: um único pacote perdido obriga o Emissor a retransmitir todos os pacotes da janela a partir do perdido. Em $N=1$, as retransmissões são o mínimo possível (uma por perda), mas o protocolo perde toda a vantagem de pipeline.

O throughput cai para a faixa de **3–21 KB/s** mesmo com $p=5\%$, uma queda de três ordens de magnitude em relação ao caso sem perda. O motivo é o temporizador de **1 segundo**: a cada perda, o Emissor aguarda 1 s antes de retransmitir. Com 62 perdas em $N=1/p=5\%$, o tempo total de timeout soma ~62 s dos ~62,2 s observados — confirmando que o throughput efetivo é controlado quase inteiramente pela frequência e duração dos timeouts.

5.4 Impacto do Tamanho da Janela N sob Perda

A tabela a seguir compara os resultados para $p=10\%$, variando apenas N :

N	Tempo (s)	Pcts. enviados	Retransmissões	Fora de ordem	Perda efetiva (%)	Throughput (KB/s)
1	113.272	1137	113	0	9.94%	9.04
2	99.244	1222	198	99	8.82%	10.32
4	122.249	1512	488	366	10.65%	8.38
8	119.029	1968	944	826	10.33%	8.60
16	131.292	3102	2078	1948	11.27%	7.80
32	110.289	4477	3453	3343	9.70%	9.28

Ao contrário do que a teoria do GBN sugeriria para enlaces com *alta latência*, o aumento de N **não melhora o desempenho** em ambiente LAN com perda simulada. Para $p=10\%$, os tempos variam de ~99 s (N=2) a ~131 s (N=16) — sem tendência clara de melhora com N maior. A explicação está no timeout de 1 segundo:

1. Em LAN, o RTT é inferior a 1 ms. A janela se enche e é drenada via ACKs em microssegundos — o canal nunca é o gargalo.
2. Cada timeout de 1 s retransmite N pacotes. Para N grande, isso significa *muito mais tráfego desnecessário*, aumentando a carga na fila de recepção e induzindo mais pacotes fora de ordem.
3. A coluna 'Fora de ordem' cresce de 0 (N=1) para 3.343 (N=32). Com N=32 e $p=10\%$, cada perda provoca a retransmissão de até 32 pacotes, dos quais 31 chegam ao Receptor fora de ordem e são descartados — gerando ACKs duplicados que retroalimentam novas retransmissões.

Esse comportamento ilustra a **ineficiência inerente ao GBN para janelas grandes em canais ruidosos**: a penalidade por perda escala com N. O Selective Repeat mitigaria esse problema permitindo a recepção e buffer de pacotes fora de ordem, retransmitindo apenas os efetivamente perdidos.

5.5 Taxa de Perda Efetiva

A taxa de perda efetiva medida pelo Receptor convergiu para a probabilidade configurada à medida que o número de pacotes aumentou, em acordo com a Lei dos Grandes Números. Para N=1, os resultados foram: 5,71% (configurado 5%), 9,94% (10%) e 21,89% (20%). Os desvios são

estatisticamente esperados para amostras de 1024 pacotes e valores distintos da semente aleatória de cada execução.

A simulação de perda incidiu **exclusivamente** sobre pacotes DATA recebidos com `seqnum == expectedSeqNum` — ou seja, em ordem. Pacotes retransmitidos fora de ordem são descartados pela lógica do GBN sem acionar o sorteio, garantindo que o mecanismo de perda não interfira com o mecanismo de descarte do protocolo, conforme a Seção 4 do enunciado.

6. Conclusão

A implementação do protocolo Go-Back-N em Java sobre UDP atingiu todos os objetivos propostos no enunciado. O Emissor e o Receptor comunicam-se exclusivamente via sockets UDP e reproduzem fielmente as FSMs descritas no Kurose & Ross, com janela deslizante, temporizador único e retransmissão em massa a partir do pacote não confirmado mais antigo.

Os testes demonstraram que o código é funcional em todos os 24 cenários testados, com integridade dos arquivos transferidos verificada via MD5 em 100% dos casos. A análise dos dados revelou três conclusões principais:

- (i) Em ambiente LAN sem perda, o tamanho da janela N tem impacto mínimo no tempo de transferência, pois a latência de propagação é negligenciável e o gargalo é o processamento local;
- (ii) Com perda, o throughput é dominado pela duração do timeout (1 s por evento de perda), e não pela largura de banda do enlace;
- (iii) Janelas maiores sob perda geram significativamente mais retransmissões desnecessárias (proporcional a $N \times \text{número_de_perdas}$), tornando o GBN com N grande ineficiente em canais ruidosos — limitação inerente do protocolo que motiva o Selective Repeat.

Como trabalho futuro, seria interessante: ajustar dinamicamente o timeout com base no RTT medido; implementar o Selective Repeat para comparação direta de desempenho; e testar em redes reais com diferentes latências e taxas de perda para validar o comportamento previsto pela teoria.

Referências

KUROSE, James F.; **ROSS**, Keith W. *Redes de Computadores: Uma Abordagem Top-Down*. 8. ed. São Paulo: Pearson, 2021. Capítulo 3, Seções 3.4 e 3.4.3.

TANENBAUM, Andrew S.; **WETHERALL**, David. *Redes de Computadores*. 5. ed. São Paulo: Pearson, 2011. Capítulo 3.